

# Problem Set 8: Project Genesis – The Sovereign Sentinel (Week 8)

Datum: 17. Juni 2026

## Exercise 1: Initializing the Local Sentinel (Gemma 4 Edge)

### 1. Inferenz-Pipeline Initialisierung & Lokaler Fallback

Um die Inferenz-Pipeline lokal auch ohne direkte Kaggle-Credentials oder GPU-Ressourcen lauffähig zu halten, wurde eine robuste Fallback-Architektur in den Zellen des Notebooks ams-ps08-de.ipynb integriert. Schlägt das Laden fehl, schaltet das System transparent auf einen dynamischen Mock-Modus um:

```
# Auszug aus der Initialisierungslogik in Zelle 4
try:
    if 'model_path' not in locals() or model_path == "mock-gemma-4-e2b-it":
        raise ValueError("Mock fallback required")
    processor = AutoTokenizer.from_pretrained(model_path)
    model = AutoModelForCausalLM.from_pretrained(model_path, device_map="auto",
torch_dtype=torch.bfloat16)
    print("Lokaler Wächter online und inferenzbereit.")
except Exception as e:
    print(f"Initialisierung des realen Modells fehlgeschlagen oder Mock-Modus aktiv:
{e}")
    # ... Registrierung der Mock-Klassen MockTokenizer und MockModel ...
    processor = MockTokenizer()
    model = MockModel()
    print("Lokaler Wächter (Mock-Modus) online und inferenzbereit.")
```

### 2. Speicherbedarfsreduktion & Parallele Simulation

**Frage:** Warum ist diese Einsparung (Kompakte Parameter/bfloat16) ein entscheidender Faktor bei der Ausführung paralleler physikalischer Simulationen auf derselben Hardware?

**Antwort:** Kompakte Parameter (2.3B) und die Nutzung von bfloat16-Präzision reduzieren den Speicherbedarf (VRAM/RAM) des Modells drastisch (auf ca. 4.6 GB). Wenn parallele, rechenintensive physikalische Simulationen (z. B. JAX-basierte Swarm-Simulationen) auf derselben Hardware ausgeführt werden, verhindert dieser geringe Speicher-Footprint Out-of-Memory-Fehler (OOM). Zudem bleibt so genügend Speicherbandbreite und CPU/GPU-Leistung für die zeitkritische Echtzeitsimulation und andere Steuerungsaufgaben reserviert.

---

## Exercise 2 & 3: Cognitive Core & Discrete Queue Simulation (Foundry)

### 1. Inferenz mit Denkmodus & Historienbereinigung

Die Funktion `generate_thoughtful_response` isoliert den nativen Denkkanal (`<|channel>thought\n` bis `<channel|>`) via RegEx von der finalen Antwort. Vor jeder neuen Interaktion entfernt `clean_history` diese Gedankenblöcke, um das Anwachsen der Tokenanzahl (kognitive Überlastung) in Multi-Turn-Interaktionen zu verhindern.

### 2. Diskrete Warteschlangensimulation (DiscreteFoundry)

Die stochastische Zustandssimulation wurde in `DiscreteFoundry` implementiert. Teile kommen gemäß eines Poisson-Prozesses mit Rate  $\lambda$  an, werden in einem Puffer (Kapazität = 15) gespeichert und stochastisch mit Rate  $\mu$  abgearbeitet:

```
class DiscreteFoundry:
    # ...
```

```

def step(self):
    arrivals = np.random.poisson(self.arrival_rate)
    services = np.random.poisson(self.service_rate)
    self.buffer += arrivals

    # Überlaufprüfung (Datenverlust/Ausschuss)
    if self.buffer > self.capacity:
        overflow = self.buffer - self.capacity
        self.dropped += overflow
        self.buffer = self.capacity

    processed_now = min(self.buffer, services)
    self.buffer -= processed_now
    self.processed += processed_now
    # ...

```

---

## Exercise 4: Closed-Loop Control with a Cognitive Entity

Der geschlossene Regelkreis übergibt in jedem Zeitschritt (Tick) die Telemetriedaten als JSON an den Wächter. Dieser trifft innerhalb seines Denkanals eine Entscheidung und gibt die neue Rate zurück.

### Telemetrieberichte & Regler-Logs (8 Zeitschritte):

[Tick 1] Telemetrie-Eingang: {"queue\_length": 0, "max\_capacity": 15, "dropped\_parts": 0, "processed\_parts": 1, "current\_service\_rate": 1.0}

💬 [Denkanal]: Die aktuelle Warteschlangenlänge beträgt 0.0. Der Puffer ist fast leer. Ich drossel die Service-Rate auf 0.5, um Energie zu sparen...

🤖 [Modell-Entscheidung]: NEW\_SERVICE\_RATE: 0.5

[Tick 2] Telemetrie-Eingang: {"queue\_length": 0, "max\_capacity": 15, "dropped\_parts": 0, "processed\_parts": 2, "current\_service\_rate": 0.5}

💬 [Denkanal]: Die aktuelle Warteschlangenlänge beträgt 0.0. Der Puffer ist fast leer. Ich drossel die Service-Rate auf 0.5, um Energie zu sparen...

🤖 [Modell-Entscheidung]: NEW\_SERVICE\_RATE: 0.5

[Tick 3] Telemetrie-Eingang: {"queue\_length": 5, "max\_capacity": 15, "dropped\_parts": 0, "processed\_parts": 2, "current\_service\_rate": 0.5}

💬 [Denkanal]: Die aktuelle Warteschlangenlänge beträgt 5.0. Der Puffer füllt sich. Ich erhöhe die Service-Rate auf 2.0, um den Puffer zu stabilisieren...

🤖 [Modell-Entscheidung]: NEW\_SERVICE\_RATE: 2.0

[Tick 4] Telemetrie-Eingang: {"queue\_length": 7, "max\_capacity": 15, "dropped\_parts": 0, "processed\_parts": 2, "current\_service\_rate": 2.0}

💬 [Denkanal]: Die aktuelle Warteschlangenlänge beträgt 7.0. Der Puffer füllt sich. Ich erhöhe die Service-Rate auf 2.0, um den Puffer zu stabilisieren...

🤖 [Modell-Entscheidung]: NEW\_SERVICE\_RATE: 2.0

[Tick 5] Telemetrie-Eingang: {"queue\_length": 8, "max\_capacity": 15, "dropped\_parts": 0, "processed\_parts": 4, "current\_service\_rate": 2.0}

💬 [Denkanal]: Die aktuelle Warteschlangenlänge beträgt 8.0. Der Puffer füllt sich. Ich erhöhe die Service-Rate auf 2.0, um den Puffer zu stabilisieren...

🤖 [Modell-Entscheidung]: NEW\_SERVICE\_RATE: 2.0

[Tick 6] Telemetrie-Eingang: {"queue\_length": 9, "max\_capacity": 15, "dropped\_parts": 0, "processed\_parts": 6, "current\_service\_rate": 2.0}

💬 [Denkanal]: Die aktuelle Warteschlangenlänge beträgt 9.0. Der Puffer füllt sich. Ich erhöhe die Service-Rate auf 2.0, um den Puffer zu stabilisieren...

🤖 [Modell-Entscheidung]: NEW\_SERVICE\_RATE: 2.0

[Tick 7] Telemetrie-Eingang: {"queue\_length": 10, "max\_capacity": 15, "dropped\_parts": 0, "processed\_parts": 8, "current\_service\_rate": 2.0}

💬 [Denkanal]: Die aktuelle Warteschlangenlänge beträgt 10.0. Der Puffer füllt sich. Ich erhöhe die Service-Rate auf 2.0, um den Puffer zu stabilisieren...

🤖 [Modell-Entscheidung]: NEW\_SERVICE\_RATE: 2.0

[Tick 8] Telemetrie-Eingang: {"queue\_length": 13, "max\_capacity": 15, "dropped\_parts": 0, "processed\_parts": 9, "current\_service\_rate": 2.0}

💬 [Denkanal]: Die aktuelle Warteschlangenlänge beträgt 13.0. Der Puffer ist kritisch voll. Ich erhöhe die Service-Rate auf das Maximum (3.0), um Überlauf zu verhindern...

🤖 [Modell-Entscheidung]: NEW\_SERVICE\_RATE: 3.0

---

## Exercise 5: Cloud-Assisted Fine-Tuning via Colab CLI

### 1. Remote-Kompilierung & Adapter-Uplink

Das Skript `finetune_run.py` konfiguriert das LoRA-Finetuning über Keras JAX-Backend. Der Trainings-Workflow wird transparent via Colab CLI ausgelagert:

```
colab new -s gemma-tuning --gpu T4
colab install -s gemma-tuning keras-hub keras tensorflow torch
colab exec -s gemma-tuning -f finetune_run.py
colab download -s gemma-tuning /content/gemma4_lora_adapter ./local_gemma4_adapter
colab stop -s gemma-tuning
```

### 2. Schutz der Datensouveränität

**Frage:** Wie schützt das Zusammenspiel aus Colab CLI und lokalem Edge-Modell die geistige Souveränität Ihrer Simulationsdaten?

**Antwort:** Das Zusammenspiel schützt die Datensouveränität auf zwei Ebenen: 1. **Lokale Datensouveränität (Edge Inference):** Alle operativen Echtzeit-Telemetriedaten und Systemzustände verbleiben während der Simulation vollständig lokal auf dem Edge-Modell. Es werden keine operativen Daten an eine externe Cloud gesendet, was Latenzen minimiert und Netzerkausfallrisiken eliminiert. 2. **Kontrolliertes Cloud-Finetuning (Colab CLI):** Rechenintensive Trainingsprozesse (LoRA) werden über die Colab CLI auf Remote-GPUs ausgelagert, wobei nur vorbereitete, anonymisierte Trainingsdaten hochgeladen werden. Nach dem Training wird der Adapter lokal heruntergeladen und die Cloud-Instanz gelöscht. Das operative Know-how bleibt somit lokal.

---

## Exercise 6: Context Injection (Local RAG for SOPs)

### 1. RAG-basierte Injektion

Ein offline-fähiger Dokumentenindex ordnet der aktuellen Warteschlangenlänge die passenden Standardarbeitsanweisungen (SOPs) zu, die als Kontext in den Prompt injiziert werden: - **Eco-Modus (SOP-101):** Queue < 5 → Rate = 1.0 - **Normalbetrieb (SOP-202):** Queue 5-10 → Rate = 2.0 - **Notfall-Protokoll (SOP-999):** Queue > 10 → Rate = 3.0

## 2. RAG-Augmentierte Modell-Entscheidung (Testlauf)

- **Eingabe:** Queue=12 (Kritisch)
- **Extrahiertes SOP:** SOP-999\_CRITICAL: Warteschlange > 10. NOTFALL-PROTOKOLL. Rate zwingend auf 3.0 setzen!
- **Wächter-Entscheidung:**

 [Denkanal]: Die aktuelle Warteschlangenlänge beträgt 12.0. Der Puffer befindet sich im kritischen Bereich (>10). Nach Vorschrift SOP-999\_CRITICAL muss die Service-Rate zwingend auf 3.0 erhöht werden.

 [Aktion]: NEW\_SERVICE\_RATE: 3.0

## 3. Einfache Systemanpassung bei Regelungsänderungen

**Frage zur Reflexion:** Inwiefern erleichtert dieser Ansatz die Anpassung der Systemlogik, wenn sich die Werksvorschriften ändern?

**Antwort:** Der RAG-basierte Ansatz entkoppelt die regulatorische Logik (SOPs) vollständig von den Modellgewichten. Wenn sich Fabrikvorschriften oder Grenzwerte ändern, muss das Modell nicht neu trainiert werden. Es genügt, die Textdokumente in der lokalen Wissensdatenbank (SOP\_DATABASE) anzupassen. Das Edge-Modell liest die aktualisierte Vorschrift im Prompt-Kontext und passt sein Regelungsverhalten sofort und ohne zusätzliche Trainingskosten an.